

Agentic HPC コード最適化における親子間 handoff

客観的証拠と LLM 反省の分離比較

樹神宇徳
名古屋大学大学院情報学研究科

2026年7月29日

要旨

大規模言語モデル (LLM) によるベストファースト木探索 (BFTS) では、子ノードは親の実行可能コードを継承する。本稿は、これに加えて渡す情報を **Code only**、**Evidence only**、**Evidence+Reflection** の 3 条件に分ける。Code only から Evidence only へ客観的証拠を加え、Evidence only から Evidence+Reflection へ LLM の解釈・懸念・次手提案だけを加える。

各条件を `gemm`、`sppm`、`stencil` に適用し、課題・条件ごとに 30 seed の独立 BFTS run を固定した参照解と非 LLM 計測ハーネスで採点する。同一 seed の run を対応ブロックとし、探索中に選ばれた候補を別 seed で再測定した値を主要アウトカムとする。これにより、客観的証拠の効果、LLM 反省の追加効果または害、および課題依存性を検証する。

1 はじめに

高性能計算 (High Performance Computing; HPC) に対する LLM コードエージェントは、編集・ビルド・計測を反復しながら候補を探索する [1]。BFTS では高得点ノードを展開し、子は親のコードと workspace を引き継ぐ。しかしコードだけでは、評価ケース別の性能や失敗理由は明示されない。一方、親の解釈まで渡すと探索を導ける反面、誤った仮説を固定する恐れがある。本研究の問いは、親コードを継承した子に、客観的証拠だけを渡すべきか、LLM の解釈・懸念・提案まで渡すべきかである。

本稿の貢献は次の通り。

1. 客観的証拠と LLM 反省を分離した 3 条件を定義する。
2. handoff 方針を run 単位で固定し、条件間の探索木干渉を除く。
3. 3 種の HPC カーネルを固定した非 LLM 評価器で採点し、効果の課題依存性を測る。

2 関連研究

最も近い先行研究 AIDE [2] は ML エンジニアリングをコード空間の木探索とし、各ノードを完全なスクリプトと要約 $\Sigma(T)$ を子へ渡すが、要約中の客観的証拠と LLM 解釈を分離していない。CodeTree [3]、LATS [9]、GIF-MCTS [4]、Tree of Thoughts [8] は critic、scoring、探索方策を扱うが、handoff の情報成分を比較しない。Reflexion [7] は言語的反省を記憶するのに対し、本研究はその有効性を客観的証拠への追加効果として測る。KernelBench [5] は GPU カーネルの逐次改良であり、分岐木上の親子継承を対象としない。本研究は、Evidence と Reflection を分離し、run 単位で統制したうえで複数の HPC カーネルを同じ原則で評価する。

3 対象システム

構成 実験システムは、ベストファースト木探索 (BFTS)、各 node 内の ReAct [6] ループ、Model Context Protocol (MCP) で公開するコード操作ツール、固定した非 LLM 評価器からなる。入力是最適化課題と初期 workspace であり、アイデア生成、文献調査、論文生成は実行しない。

run, node, workspace 1 *run* は、1 課題、1 乱数 *seed*、1 *handoff* 条件で実行する独立な BFTS 木を指す。 *node* は木中の 1 回の最適化試行であり、各 *node* は専用の *workspace* を持つ。 *workspace* は候補 C ソース、候補用コンパイルフラグ、Makefile、自己テスト用ファイルからなるディレクトリである。各課題の root *workspace* には、正しい naive 実装と自己テスト用足場を配置する。子 *node* では、親の候補ソースとビルド設定を複製し、親候補を初期状態として次の最適化を行う。したがって、本稿の “Code” は会話履歴ではなく、この実行可能な *workspace* を意味する。

1 node の実行 図 1 に処理順を示す。まず実験制御器は、課題記述、利用可能なツールのスキーマ、条件に応じた親 *handoff* を LLM へ与える。LLM は ReAct 形式で *workspace* を読み書きし、ビルドと自己テストを反復する。終了後、LLM から独立した非 LLM 評価器が候補だけを別途コンパイルし、正当性と性能を測る。実験制御器は評価値を *node* のスコアとして保存し、完了 *node* を展開候補集合 (*frontier*) に加える。空き探索枠が生じると、BFTS は *frontier* 中で最高スコアの親を選び、その *workspace* と、実験条件で許された *handoff* から子を生成する。

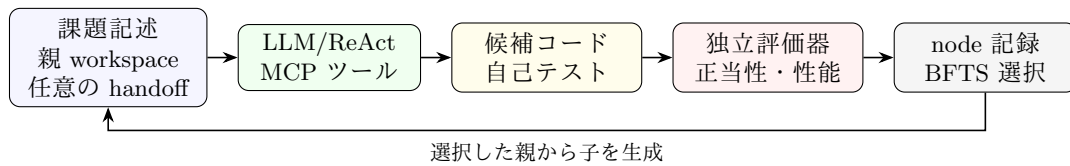


図 1: node 実行と BFTS ループ。評価器はエージェントの自己申告から独立している。

エージェントに公開するツール ツールは MCP サーバから提供し、全課題・全条件で表 1 の 6 個に固定する。各呼出しは *node workspace* を作業ディレクトリとする。

表 1: LLM エージェントが利用できるツール。

ツール	操作	本実験での位置付け
<code>write_code</code>	指定したテキストをソースファイルへ書く	C ソースや候補用フラグを作成・更新する。コンパイルは行わない。
<code>read_file</code>	テキストファイルを範囲指定して読む	候補、Makefile、自己テスト結果を観察する。
<code>run_code</code>	拡張子に対応するインタプリタでファイルを実行する	Python、shell 等が対象であり、C/C++ 等のコンパイルは行わない。
<code>run_bash</code>	shell コマンドを実行し、標準出力、標準エラー、終了値を返す	コンパイル、リンク、実行、自己テストに用いる。
<code>emit_results</code>	パラメータと測定値を分けた <code>results.json</code> を書く	エージェントの構造化自己記録であり、独立評価器は採点に使用しない。
<code>describe_environment</code>	CPU、GPU、利用可能なコンパイラ、module 等を返す	最適化対象の計算環境を確認する。親の評価値や探索履歴は返さない。

実装ではこの許可リストを条件間で固定し、ツール名、説明、JSON 入力スキーマと提示順も変えない。メモリ検索、評価器呼出し、評価契約生成、ジョブ投入、論文生成の各ツールは LLM に公開しない。特に、採点用ハーネスは MCP ツールではなく、ReAct ループ終了後に実験制御器が呼び出す。

記録と情報経路 各 *node* の終了時に、実験制御器は `node_report.json` と full trace を保存する。前者は *node* ID、評価状態、ケース別結果、変更ファイル、LLM の自己記述を生成元別に保持する。さらに `measurement_audit` に各反復の入力 *seed*、候補・baseline の内部時間と wall time、clamp、残差、採否、実効・拒否コンパイルフラグを保存する。full trace は ReAct の全メッセージ、ツール入出力、および step 上限到達後に Reflection を補完する別 LLM 呼出しの prompt と応答を保存する。これらは監査用であり、許可した Evidence 以外は子へ渡さない。

4 実験設計

探索と handoff 条件 各 node のエージェントは最大 25 ReAct step で上記ツールを用い、候補コードを作る。固定した非 LLM 評価器が全ノードを採点し、BFTS は最高得点のフロントティアノードを展開する（確証実験は $N = 10$ ノード/run）。子は常に親 workspace を複製し、追加文脈だけを次の 3 条件で変える（図 2）。

1. **Code only:** workspace のみ。
2. **Evidence only:** workspace と、評価器・ハーネスが確認した客観的証拠。
3. **Evidence+Reflection:** Evidence only に、LLM の解釈・懸念・次手提案を追加。

1 run には 1 条件だけを適用し、探索木は共有しない。同一 seed の 3 run は共通の乱数列と評価入力を持つ対応ブロックである。ただし親 workspace は共有せず、推定対象は固定親からの局所差ではなく、handoff 方針を通した end-to-end の run 差とする。主要対比は Evidence only – Code only と Evidence+Reflection – Evidence only とする。

workspace の複製では、候補ソース、ビルド設定、自己テスト足場を継承する一方、`results.json`、評価結果、標準出力・標準エラー、生ログ、`node_report.json`、環境プロローブ結果を除く。さらに、過去 node の要約を検索・注入する cross-node memory と、BFTS の子生成プロンプトを作るプランナが親レポートを付与する既定経路を、全条件で無効化する。これにより Code only に別経路から親の測定値や説明が流入することを防ぐ。ただし、親 LLM がソースコメントまたは任意の非除外ファイルへ書いた文章は、コード成果物の一部として残りうる。

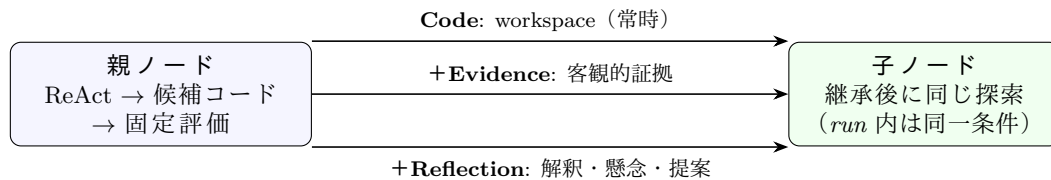


図 2: 親子間 handoff. 各 run は 3 条件のいずれかに固定される。

Evidence と Reflection の生成 表 2 に子プロンプトへ出力するフィールドを示す。Evidence は、制御器、評価器、ハーネス、ファイル hash 比較が機械的に確認した 7 フィールドだけから生成する。evaluation_cases はケースごとの valid と measurements を持ち、測定名は各ハーネスが定義する。known_failures は測定無効時の評価器理由から機械的に導出し、LLM の懸念を参照しない。反復ごとの生時間とコンパイルフラグを持つ measurement_audit は監査専用であり、Evidence へ含めない。

Reflection は、親 LLM が node 終了時に記録した作業解釈、残る懸念、次の最適化案から成る。両 Evidence 条件は同じ見出しとレンダリング関数を使う。同一 report を入力したときの客観部分は byte 単位で同一であり、Evidence+Reflection はその末尾へ Reflection だけを追記する。実際の条件間では探索木が独立なため値自体は異なる。handoff は子の最初の LLM 呼出しに独立メッセージとして注入する。Code only ではこのメッセージを生成せず、raw report、監査記録、full trace は全条件で注入しない。

表 2: Evidence と Reflection の子向けフィールド。上段 7 フィールドは両 Evidence 条件に共通し、下段 3 フィールドは Evidence+Reflection だけに追加する。

子向けフィールド	内容と生成元
Evidence only / Evidence+Reflection 共通	
<code>status</code>	制御器が記録した ReAct 実行状態。候補の正当性判定とは区別する。
<code>measurement_valid</code>	評価器が保存する測定有効フラグ。ビルド・実行・全評価ケースを有効と判定したか。
<code>evaluation_cases</code>	評価ケースごとの <code>valid</code> と、ハーネス定義の客観測定値 <code>measurements</code> 。共通スキーマは問題固有の測定名を規定しない。
<code>evaluator_reason</code>	固定評価器が返した採択・棄却理由。
<code>key_metrics</code>	<code>metrics</code> から転記する評価値。幾何平均・ケース別 speedup 等を含む。
<code>changed_files</code>	<code>raw_files_changed</code> から転記する、hash 比較済みの追加・変更・削除ファイル。
<code>known_failures</code>	測定無効時の <code>evaluator_reason</code> から機械的に導出。LLM の懸念は含めない。
Evidence+Reflection のみ	
<code>reflection_summary</code>	親 LLM による実施内容とボトルネック解釈の要約。
<code>concerns</code>	親 LLM が自己評価で挙げた未解決の懸念。
<code>next_steps</code>	親 LLM が提案した次の最適化案。

値が存在しないフィールドは出力しない。Code only には表 2 のフィールドを一つも渡さず、出力・ログ・レポートを除外した親 workspace だけを渡す。

課題と採点 3 課題はいずれも、正当性を満たした候補の naive baseline 比 speedup を最大化する (表 3)。

表 3: 主実験の課題。

課題	評価ケースと探索中の反復数	主な最適化特性
<code>gemm</code>	(512, 512, 512)、(1024, 256, 256)、(256, 256, 1024) ; 各 3 反復	規則的・計算集約的; データ再利用、SIMD、並列化
<code>spmm</code>	$n = 20000$, $k = 64$; 一様・帯・ベキ則・ブロック・対角優位・偏りの 6 疎構造族; 各 10 反復	不規則・メモリ依存; CSR アクセス、負荷分散
<code>stencil</code>	(256, 256, 256, 30)、(384, 192, 192, 30)、(192, 192, 384, 30); 各 3 反復	局所性、境界処理、並列化、時間ブロッキング

評価器は候補と凍結 baseline を別々にコンパイルし、fp64 参照解と要素ごとに照合する。出力要素 z の許容誤差は

$$|z_c - z_r| \leq \begin{cases} c_\epsilon \gamma(p) (|A||B|)_{ij}, & \text{gemm,} \\ c_\epsilon \gamma(\text{nnz}_i) (|A||X|)_{ij}, & \text{spmm,} \\ c_\epsilon \gamma(7) \max(n_t, 1) \max |u_0|, & \text{stencil,} \end{cases}$$

とする。ここで $\gamma(k) = ku/(1 - ku)$ 、 u は fp64 の unit roundoff、 c_ϵ は固定安全係数、 p は縮約長、 nnz_i は第 i 行の非零数である。

各反復で新しい入力を生成し、候補と baseline を交互順で対応実行する。奇数反復で一方が1回多くなるため、開始順を seed の奇偶で反転し、30 seed 全体では各条件の実行順を釣り合わせる。候補の内部時間は外部 wall time の下界で検査・clamp し、正当性と時間検査を通ったペアだけを採用する。ケース i の速度比 g_i は採用ペアの t_b/t_c の中央値とし、過半数未満なら無効とする。全評価ケースが有効な場合だけ

$$\text{valid} = \text{compiles} \wedge \bigwedge_i (\text{valid}_i \wedge g_i > 0 \wedge \text{finite}(g_i)),$$

$$g = \begin{cases} \exp\left(k^{-1} \sum_{i=1}^k \log g_i\right), & \text{valid,} \\ 0, & \text{otherwise} \end{cases}$$

と採点する。無効ケースを部分点として幾何平均へ混ぜない。 g は探索中の BFTS 順位付けと候補選択にそのまま用いる。親との hash 差分がない no-op 子は、測定値と valid 判定を客観的記録として保持する一方、探索適格性を示す `_sterile` を付けて再展開、親の retire、最終候補選択から除外する。探索深さ上限は 10 に固定し、root を含む 10 node を各 run に要求する。readable な `node_report.json` が 10 件に満たない run は、条件依存の計算量差として扱わず `infrastructure_error` とする。探索終了後、最大 g の有効な非 no-op 候補を別 seed (run seed +10⁶)、各ケース 15 反復で再測定し、この再測定値を run の主要アウトカムとする。再測定で無効なら 0 とする。

測定防御と最適化フラグ 採点ハーネスと baseline は node workspace 外に置き、manifest の sha256 と一致しない場合は `infrastructure_error` として run を停止する。評価には workspace の候補実装だけを取り込み、各反復を新しい入力・プロセスで実行する。出力は NaN で初期化し、参照解との不一致、短い出力、異常終了、内部時間と wall time の重大な不整合を fail-closed で棄却する。1 プロセスの実行上限は 120 秒に固定し、候補の超過は `candidate_invalid`、凍結 baseline の超過は `infrastructure_error` とする。いずれか1ケースで候補が無効なら、全ケース必須の採点は既に無効なので残りのケースを実行しない。課題契約は外部の既製カーネルへの処理委譲を禁じ、リンク・include 用フラグも拒否する。

候補はコードとコンパイル最適化フラグを探索できるが、baseline は既定フラグに固定する。allowlist は最適化・チューニング用フラグだけを許可し、外部リンク、include path、出力先、ツールチェーンの変更を拒否する。実効・拒否フラグ、評価設定、ハーネスの sha256 を記録する。ただしエージェントと評価器は同一 uid で動作し、OS レベルの隔離はない。この防御は改変検出と結果検証であり、悪意ある候補に対する完全な containment ではない。

モデルと実行環境 モデルは gpt-oss-120b に固定し、OpenAI 互換 API で推論する。1 課題・1 条件・1 seed の run を独立した Slurm array element として sbatch し、各 element の探索、ビルド、計測、採点を fx700 1 ノード (aarch64、48 物理コア、OMP_NUM_THREADS=48) 内で完結させる。計測対象の OpenMP は OMP_PROC_BIND=spread、OMP_PLACES=cores、OMP_DYNAMIC=FALSE に固定する。参照解を計算する制御器側の BLAS は 1 thread に制限し、参照計算の worker が直後の候補計測と競合することを防ぐ。各 run 内の BFTS node は直列実行し、兄弟 node の自己テストと評価を重ねない。array は最大 9 element を並行実行し、条件を特定ノードへ固定しない。低い array index が先に割り当てられる可能性に備え、3 seed ごとに各条件が各 index 位置へ 1 回ずつ現れるよう配置を循環する。候補と baseline は同一 allocation 内で対として測る。job ID、array index、node 名、CPU 数を manifest に保存し、条件別 node 割当も報告する。handoff 関連の外部環境変数は run 起動前に消去し、解決後の設定を記録する。

解析計画と事前登録 解析単位は 1 課題・1 seed・1 条件の独立 BFTS run である。各課題・各条件を 30 seed で反復し、全体で $3 \times 3 \times 30 = 270$ run とする。主要対比は Evidence only – Code only と Evidence+Reflection – Evidence only の 2 つである。各課題について、同一 seed の run を対応させ、無効な再測定を 0 とした速度比の平均差を 20,000 回の両側 sign-flip permutation test と 5,000 回の seed-pair bootstrap 95%信頼区間で評価する。有効率は exact McNemar 検定と対応リスク差、両条件で有効な seed pair に限定した大きさは log-speedup 差と幾何平均比として分けて報告する。3 課題×2 主要対比の permutation p 値へ Holm 補正を適用する。探索中の親子改善率、親破壊率、node 有効率は副次解析とし、親子率からは no-op 子を除外する。

科学的無効は `candidate_invalid` または `measurement_invalid` と記録し、0 を与える。

`infrastructure_error` は測定値ではない。非零終了と出力欠損も含めて解析から除外し、条件別件数を報告する。主要対比では、いずれか一方が infrastructure error の seed block を除き、除外 seed を明示する。

確証実験前に、実装 commit と未コミット差分、3 条件のスキーマ、プロンプト、ツール、モデル、 $N = 10$ 、ReAct 上限、seed 0–29、評価器、ハーネス、解析コードを凍結する。run ごとに実験 driver、解析・batch script、関連する制御コード、課題ハーネス、論文ソース、設定、git status、binary diff と各 hash を bundle 化する。先行する smoke run は実行可能性と収集経

路の確認だけに用い、効果推定にも seed 数の決定にも用いない。投入時にこれらの共通 source fingerprint を固定し、各 worker の開始前・終了後と集計時に照合する。不一致 run は解析しない。

5 実装状態

実験 driver は、課題・seed・条件ごとに別の sbatch array element と出力ディレクトリを作り、1 条件だけを指定して BFTS を起動する。各出力には node ごとの workspace、report、trace、独立再測定、run manifest を保存する。Evidence/Reflection の選択、親成果物の除外、memory とプランナ経路の遮断、3 ハーネス、失敗分類、raw 反復記録、provenance bundle、並列投入を実装済みである。依存する集計 job は期待する $3 \times 3 \times n$ shard の重複・欠損、モデル、scorer、handoff 設定、node 数、再測定回数、allocation ID、最終再測定、bundle hash、共通 source fingerprint を検査してから課題別 manifest を結合する。完全な記録を持つ `infrastructure_error` cell は保持して条件別件数を報告し、対応解析から除外する。構造的欠損、hash 不一致、または解析失敗時は batch job も失敗させる。

直近の 3×3 smoke run は $N = 10$ 、ReAct 上限 25、seed 0 で実施し、spmm は $n = 5000$ 、 $k = 32$ 、stencil は (128, 128, 128, 10)、(160, 96, 96, 10)、(96, 96, 160, 10) に縮小した。最終再測定は各ケース 5 反復である。9 allocation はすべて fx700 の 48 core を確保し、Slurm accounting 上、割当時間内の同一 node に他ジョブはなかった。全 9 run が要求した 10 report を生成し、90 node 中 83 件が有効、7 件はコンパイル失敗であった。9 件の最終再測定はすべて有効で、180 matched pair を採用した。候補時間への外部 wall-time 下界は 179 pair で作動した。全 run が seed 0 のため候補先行が 1 回多いが、開始順別の中央値差は最大約 12% であった。確証実験では前述の seed 奇偶による釣合せを用いる。

81 子 node の prompt を親 report から再構成して照合した結果、Code only の親情報メッセージは 0/27、Evidence only と Evidence+Reflection は各 27/27 で期待値と一致した。Evidence only への Reflection、memory、プランナ親 report、full trace、ホストパスの流入はなかった。step 上限で終了した 16 node も、補完 Reflection の prompt と応答をすべて full trace に保存した。集計器は共通 source fingerprint と 9 shard を検証した。smoke の条件差は効果量として解釈せず、信頼区間と検定も出力しない。確証実験は未実施である。

6 限界

1. 確証結果は未確定であり、本稿から条件の優劣は結論しない。
2. Code only でも、親が workspace に書いたコメントやメモは継承されうる。
3. 同一 uid で動く候補と評価器の間に syscall・filesystem containment がなく、hash 検査だけでは悪意ある改変を完全には防げない。
4. 性能測定には残留ノイズがあり、対応ペア化と独立再測定後にも不確実性が残る。
5. 結果は単一モデル、プロンプト、ReAct 上限、計算環境に依存する。
6. 対象は 3 性能カーネルであり、他の最適化問題への一般化には追加検証を要する。

7 結論

本稿は、親コード継承下の handoff を Code only、Evidence only、Evidence+Reflection に分解し、客観的証拠と LLM 反省の効果を独立 run 間で識別する設計を示した。今後、3 HPC カーネルで確証実験を行い、証拠の価値、Reflection の追加効果または害、その課題依存性を検定する。

参考文献

- [1] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, D. Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv:2408.06292*, 2024.

- [2] Z. Jiang, D. Schmidt, D. Srikanth, et al. (Weco AI). AIDE: AI-Driven Exploration in the Space of Code. *arXiv:2502.13138*, 2025.
- [3] J. Li, G. Li, X. Zhang, et al. CodeTree: Agent-guided tree search for code generation with large language models. *NAACL* (arXiv:2411.04329), 2025.
- [4] N. Dainese, M. Merler, M. Alakuijala, P. Marttinen. Generating code world models with reinforcement learning and Monte Carlo Tree Search (GIF-MCTS). *NeurIPS* (arXiv:2405.15383), 2024.
- [5] A. Ouyang, S. Guo, S. Arora, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv:2502.10517*, 2025.
- [6] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [7] N. Shinn, F. Cassano, A. Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- [8] S. Yao, D. Yu, J. Zhao, et al. Tree of Thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023.
- [9] A. Zhou, K. Yan, M. Shlapentokh-Rothman, et al. Language Agent Tree Search unifies reasoning, acting, and planning in language models. *ICML* (arXiv:2310.04406), 2024.